LAB PRACTICE

P4. A ring of genetic algorithm subpopulations

One genetic algorithm per process, an individual travelling round the ring every few generations, and eight functions to write.

The problem

Maximise `x*x + y*y`. Each variable is written in 16 bits and read as a plain unsigned integer, so both run from 0 to 65535 and the largest value of the function is 2 × 65535 × 65535 = 8 589 672 450. Every process holds a subpopulation of 64 individuals kept in order from best to worst. In each generation two parents are chosen by tournament, crossed, mutated, evaluated and placed in the subpopulation, which keeps its size. Every 20 generations each process sends one individual to the next process in the ring and receives one from the previous process. When every process has done 20 000 evaluations, process 0 reports the best value of the ring and the number of evaluations after which the maximum was first reached.

What you are given

File	What it is	State
`P4_ga_ring.c`	The constants, the ring, the migration exchange, the main loop, the stop condition and the final report.	Given and working

There are no deliberate defects in this file. If it does not work, the reason is in a function you wrote.

What you have to write

Eight functions with empty bodies. The comment above each one is the specification. How you write it is your decision.

Function	What it must produce
`decode`	The value of one variable, read from its bits
`fitness`	The value of the function for one individual
`tournament`	The index of the winner of a binary tournament
`crossover_two_point`	Two children from two parents
`mutate`	The same individual with some bits flipped
`insert_sorted`	The subpopulation with one individual placed in order
`migrate_select`	The individual this process will send
`migrate_accept`	The subpopulation after considering the one received

Building and running

```
mpicc -Wall -O2 -o P4_ga_ring P4_ga_ring.c     # -Wall must print no warnings
mpirun --oversubscribe -np 4 ./P4_ga_ring 1      # four processes; the last number is the seed
mpirun --oversubscribe -np 4 ./P4_ga_ring 2      # same seed, same run; another seed, another run
```

Every run prints one line per process and then, from process 0, the best value of the ring, the largest value that exists, and the number of evaluations after which the maximum was first reached, or that it was not reached. That number is what stages two and three measure.

Where to start

Build it before writing anything. It compiles and runs, and reports a best value of zero. Write the functions in the order they appear: without `decode` and `fitness` nothing else can be tested. Your job is to make the best value rise to the maximum, which the program prints beside it.

Hand in three separate files, not compressed

`P4_ga_ring.c`, `results.txt` and `evidence_NAME_SURNAME_ddmmyy.pdf`. Use exactly these names. If you only complete stage one, hand in the three of them with what stage one asks for. The table at the end of the next page says what goes inside each file.

A note on how you work

You may use any tool to understand the code. You may not hand in anything you cannot explain, and you can be asked in class to justify any line.

THE WORK

Three stages

Stage one is compulsory. Stages two and three start from it and change one thing at a time. Every run below is five runs, with seeds 1 to 5; record for each one the number printed by process 0, or «not reached».

STAGE 1	Make it find the maximum
Start from	`P4_ga_ring.c` as you received it.
What to do	Write the eight functions, in the order they appear, building after each one until `-Wall` prints no warnings.
What to observe	Each process reports its best individual, and process 0 reports the best of the ring beside the largest value that exists, and after how many evaluations it was first reached.
What to hand in	`P4_ga_ring.c` completed; in `results.txt`, the output of one run with four processes and seed 1.

STAGE 2	The ring: number of processes and migration period
Start from	Your completed `P4_ga_ring.c`.
What to do	Run with 1, 2, 4 and 8 processes, seeds 1 to 5 each. Then fix four processes and set `MIGRATE_EVERY` to 5 and to 100, seeds 1 to 5 each; 20 is the run you already have.
What to measure	For each row: the five numbers, their mean, and how many of the five reached the maximum. Multiply the mean by the number of processes to get the evaluations of the whole ring.
What to hand in	Two tables in `results.txt`. One sentence on what happens to the evaluations of one process and to the evaluations of the whole ring when processes are added, and one on what a very short and a very long migration period do.

STAGE 3	What travels, and the size of the subpopulation
Start from	Your completed `P4_ga_ring.c`, four processes, `MIGRATE_EVERY` back to 20.
What to do	Set `MIGRATE_BEST` to 0, so a random individual travels instead of the best, seeds 1 to 5. Put it back to 1 and run with `POP_SIZE` 16 and with `POP_SIZE` 256, seeds 1 to 5 each.
What to observe	The same measure as stage two, for the three rows, against the row with four processes of stage two.
What to hand in	One table with three rows in `results.txt`, one sentence on what sending a random individual changes, and one on what a small and a large subpopulation change.

What you hand in: three files, exact names

File	Stage	What it contains
`P4_ga_ring.c`	1	The completed program, compiling without warnings with `-Wall`, with the constants as you received them: `MIGRATE_EVERY` 20, `MIGRATE_BEST` 1, `POP_SIZE` 64.
`results.txt`	1 to 3	Plain text copied from your terminal, in three parts headed exactly STAGE 1, STAGE 2 and STAGE 3. Stage 1: the output of the run with four processes and seed 1. Stage 2: the two tables and the two sentences. Stage 3: the table and the two sentences.
`evidence_NAME_SURNAME_ddmmyy.pdf`	1 to 3	One PDF with one handwritten page, scanned, with your name and the date: one short paragraph for each of which function was hardest and why, what you changed in stages two and three, and what you measured.

If you want to go further

The migration is synchronous: every process waits at the exchange for its two neighbours. A non blocking exchange, with `MPI_Isend`, `MPI_Irecv` and `MPI_Wait`, lets a process go on evolving while the message travels. Not required. If you do it, hand it in as a fourth file, `P4_ga_ring_async.c`, and say in `results.txt` whether the measure of stage two changes.